Report File

THE ASSIGNMENT ENTERED IN THIS REPORT HAVE BEEN SATISFACTORILY PERFORMED BY

Registration No: 25BAI11372
Name of Student: Hafizur Rahman
Course Name: Fundamental in AI and ML
Course Code: CSA2001
School Name: SCAI
Semester: Winter 2025/2026
Course Faculty Name: Dr. Bandla Pawan Babu

1. Title

SmartChat AI – A Voice-Enabled AI Chatbot using Flask and Groq API

---

2. Abstract

SmartChat AI is a full-stack web-based chatbot application that enables users to interact with an AI assistant through both text and voice. The system is built using Flask for the backend and HTML, CSS, and JavaScript for the frontend. It integrates a high-performance AI model via the Groq API to generate intelligent responses in real-time. Additionally, the application supports browser-based voice input and speech output, enhancing user interaction and accessibility.

---

3. Objectives

- To build a real-time AI chatbot using modern web technologies
- To integrate voice input and output for better user experience
- To store chat history using a lightweight database
- To create a simple and interactive user interface
- To use a fast and free AI inference API

---

4. Technologies Used

- **Backend**
  - Python
  - Flask
- **Frontend**
  - HTML
  - CSS
  - JavaScript
- **AI Integration**
  - Groq API (LLaMA models)
- **Database**

- SQLite

◆ **Browser APIs**

- Web Speech API (Speech Recognition & Speech Synthesis)

---

5. System Architecture

Components:

1. **Frontend (Client Side)**
   - User interface
   - Sends messages to backend
   - Handles voice input/output
2. **Backend (Server Side)**
   - Flask server
   - Processes requests
   - Connects to AI API
3. **Database**
   - Stores previous chat messages
4. **AI API**
   - Generates intelligent responses

---

6. Working of the System

1. User enters text or uses voice input
2. Frontend sends request to Flask backend (/chat)
3. Backend processes input and sends it to Groq API
4. AI generates response
5. Response is sent back to frontend
6. Frontend displays and optionally speaks the response

---

7. Features

- Real-time chat system

- Voice input using browser
- Voice output (text-to-speech)
- AI-powered responses
- Chat history storage
- Fast response time
- Simple UI

---

8. Testing

Functional Testing

- Chat input/output working correctly
- Voice input recognized properly
- AI responses generated successfully

Error Handling

- Handles empty input
- Displays error messages for API issues

---

9. Challenges Faced

- API quota limitations with initial setup
- Deprecated models in AI API
- Microphone issues using Python libraries
- Managing asynchronous frontend requests

---

10. Solutions Implemented

- Switched to Groq API (free and faster)
- Updated to latest supported models
- Replaced Python voice with browser-based voice
- Added error handling for smooth experience

---

11. Future Enhancements

- User login and authentication
- Chat export/download feature
- Dark mode UI
- Mobile responsiveness improvements
- Deployment on cloud (Render, AWS, etc.)
- Multi-language support

---

12. Conclusion

SmartChat AI successfully demonstrates how modern web technologies can be combined with AI to create an interactive chatbot. The integration of voice features enhances usability, while the use of a fast AI API ensures real-time performance. This project serves as a strong foundation for building more advanced AI-powered applications.

---

13. References

- Flask Documentation
- Groq API Documentation
- Web Speech API (MDN Docs)
- SQLite Documentation

---

14. Project Outcome

- Fully working chatbot
- Voice-enabled interaction
- Real-time AI responses
- Clean and simple UI